Technical Report: Semantic Duplicate & Near-Plagiarism Detection Engine

1. Project Overview

This project implements a command-line engine for detecting duplicate, near-duplicate, and semantically similar documents.

The system compares two main approaches:

1. Shingling + MinHash + LSH
2. TF-IDF Weighted SimHash

The goal is to build a reproducible and modular CLI-based project, not a notebook-only implementation.

2. Problem Definition

Given a collection of documents, the task is to identify pairs of documents that are highly similar.

A naive approach compares every document with every other document. For n documents, this requires:

$$n(n - 1) / 2$$

comparisons.

This becomes computationally expensive for large document collections. Therefore, approximate methods such as MinHash, LSH, and SimHash are used to reduce computation while preserving similarity information.

3. Text Preprocessing

Each input document is first cleaned and converted into a normalized representation.

Implemented preprocessing steps:

- Unicode normalization
- Persian/Arabic character normalization
- lowercasing
- HTML tag removal
- URL and email removal
- punctuation and symbol removal
- extra whitespace removal
- word-level tokenization
- stopword removal
- word shingling

The default shingle size is:

k = 3

For very short or empty documents, the system marks the document as invalid and avoids unreliable similarity computation.

4. Word Shingling

A word shingle is a sequence of k consecutive words.

Example:

text = "lion ate zebra and goat"

k = 2

The generated shingles are:

(lion, ate)

(ate, zebra)

(zebra, and)

(and, goat)

Shingling preserves local word order. This is important because two documents may contain the same words but in different orders.

5. Exact Jaccard Similarity

After shingling, each document is represented as a set of shingles.

For two shingle sets A and B, Jaccard similarity is:

$J(A, B) = |A \cap B| / |A \cup B|$

This method is accurate but expensive when applied to all document pairs in a large corpus.

6. MinHash

MinHash creates a compact signature for each document's shingle set.

Instead of comparing large shingle sets directly, the system compares shorter MinHash signatures.

In this implementation:

num_perm = 128

The estimated similarity between two documents is computed as the fraction of equal positions in their signatures.

MinHash was implemented from scratch using deterministic hash functions. Python's built-in hash function was avoided because it is randomized between program runs.

7. Locality Sensitive Hashing

LSH is used to reduce the number of pairwise comparisons.

The MinHash signature is divided into multiple bands.

Default setting:

num_bands = 32

Two documents become candidate pairs if they share at least one bucket in at least one band.

The final exact Jaccard comparison is then performed only on candidate pairs, not on all possible document pairs.

8. TF-IDF Weighted SimHash

SimHash creates a compact fingerprint for each document.

The implemented method uses:

- word tokens
- normalized term frequency
- smoothed inverse document frequency
- 64-bit deterministic hashing
- Hamming distance

Default setting:

hash_bits = 64

For each token, its TF-IDF weight contributes positively or negatively to each bit position. The final fingerprint is created from the sign of each accumulated bit dimension.

Similarity is computed as:

similarity = 1 - hamming_distance / hash_bits

9. CLI Commands

The project provides three main CLI commands.

9.1 Compare Two Documents

```
python -m plagiarism_engine.cli compare \
--file-a data/sample_corpus/doc_01.txt \
--file-b data/sample_corpus/doc_02.txt \
--shingle-size 3 \
--output outputs/two_file_compare.json
```

This command computes:

- exact Jaccard similarity
- MinHash similarity
- SimHash fingerprints
- SimHash Hamming distance
- SimHash similarity

9.2 Search Similar Documents in a Corpus

```
python -m plagiarism_engine.cli corpus \
--data data/sample_corpus \
--threshold 0.1 \
--shingle-size 3 \
--output outputs/candidates.csv
```

This command uses:

- preprocessing
- shingling
- MinHash
- LSH
- final Jaccard filtering

The output file contains candidate document pairs and LSH reduction statistics.

9.3 Evaluate on Labeled Pairs

```
python -m plagiarism_engine.cli pairs \
--pairs data/sample_corpus/sample_pairs.csv \
--text-col-a text_a \
--text-col-b text_b \
--label-col label \
--threshold 0.1 \
--simhash-threshold 0.65 \
```

--shingle-size 2 \

--output outputs/metrics.csv

This command computes evaluation metrics for:

- exact Jaccard
- MinHash
- SimHash

10. Evaluation Metrics

The following metrics are implemented:

- accuracy
- precision
- recall
- F1-score
- runtime

For binary duplicate detection:

prediction = 1 if similarity >= threshold

prediction = 0 otherwise

Precision measures how many predicted duplicate pairs are actually duplicates.

Recall measures how many true duplicate pairs are successfully detected.

F1-score balances precision and recall.

11. Sample Dataset

A small sample dataset is included in:

data/sample_corpus/

It contains:

- sample text documents
- a small labeled pair CSV file

These files are used for testing the CLI and generating reproducible outputs.

The current sample evaluation output is stored in:

outputs/metrics.csv

12. Current Results

The current implementation generates metrics for three methods:

1. exact Jaccard
2. MinHash
3. SimHash

The results are available in:

outputs/metrics.csv

Because the current sample dataset is very small, these results should be interpreted only as a functional demonstration. For final evaluation, a larger labeled dataset such as Quora Question Pairs should be used.

13. Error Analysis Plan

For the final report, error analysis should include:

- false positives: non-duplicate pairs predicted as duplicates
- false negatives: duplicate pairs missed by the system

Expected causes of false positives:

- too small shingle size
- generic common words
- low similarity threshold

Expected causes of false negatives:

- heavy paraphrasing
- very short texts
- too large shingle size
- different vocabulary with similar meaning

14. Reproducibility

The project is reproducible through:

- fixed random seed for MinHash
- deterministic hashing using hashlib
- command-line execution
- saved CSV and JSON outputs
- automated tests

Run all tests with:

pytest tests

15. Conclusion

This project implements a complete CLI-based near-duplicate detection engine.

The MinHash + LSH pipeline is suitable for scalable candidate generation, while SimHash provides a compact fingerprint-based method for fast similarity estimation.

The final system is modular, testable, and suitable for further evaluation on larger datasets.

16. Pair-Level Error Analysis Output

To support error analysis, the system produces an optional pair-level prediction file:

outputs/pair_predictions.csv

This file is generated by the `pairs` CLI command using the `--details-output` argument.

Each row contains:

- pair id
- first text
- second text
- true duplicate label
- exact Jaccard similarity
- MinHash similarity
- SimHash similarity
- prediction of each method
- error indicator for each method

This output allows manual inspection of false positives and false negatives.

A false positive occurs when the true label is 0 but the method predicts 1.

A false negative occurs when the true label is 1 but the method predicts 0.

For the final report, several rows with error value equal to 1 should be selected and discussed. This helps explain the limitations of each method.

Possible causes of false positives:

- low threshold
- very common words
- short documents
- small shingle size

Possible causes of false negatives:

- paraphrasing with different vocabulary
- very short input texts
- large shingle size
- semantic similarity without lexical overlap